

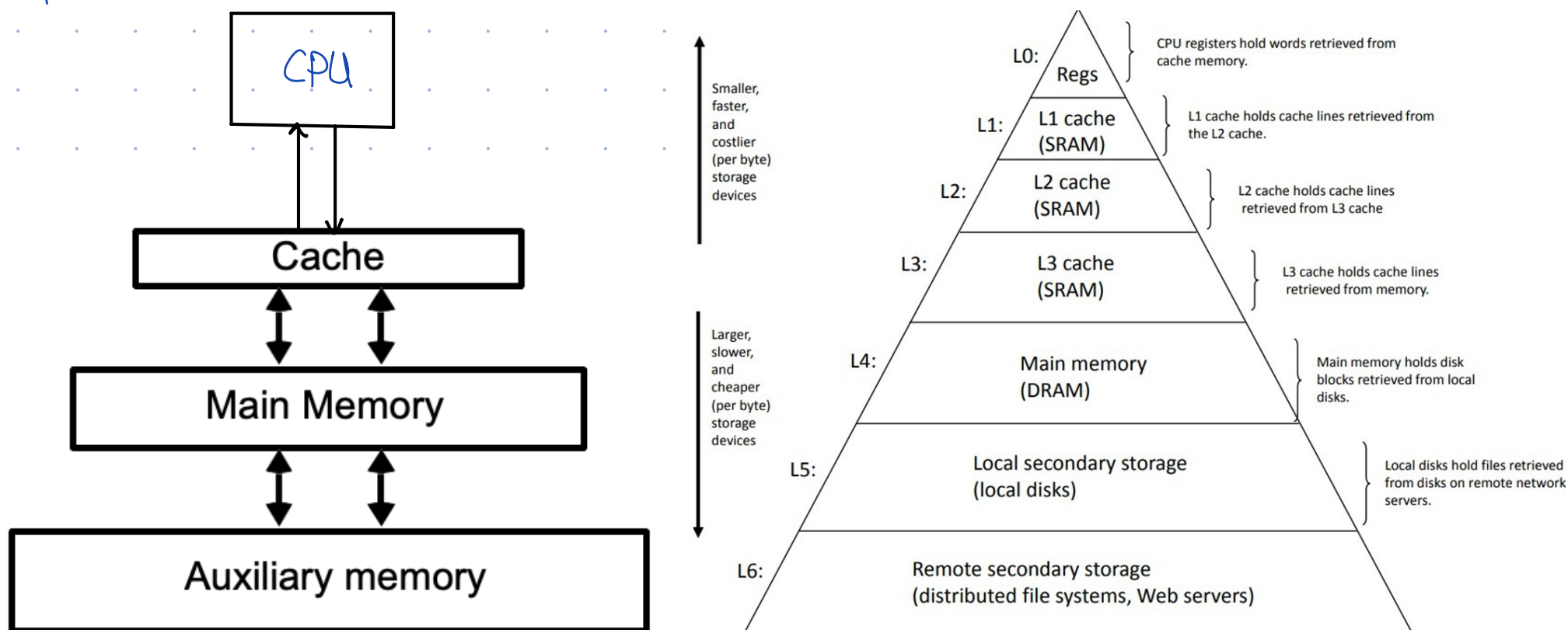
Memory is a major factor in determining performance.

Principle of locality

States that a processor accesses relatively small portions of programs at any instant time.

Memory Hierarchy:

Consists of multiple levels of memory with different speeds & sizes.



As the distance from the processor increases, so does size of memory.

Why do we need memory hierarchy?

1. To obtain the highest possible access speed while minimizing the total cost of the memory system.

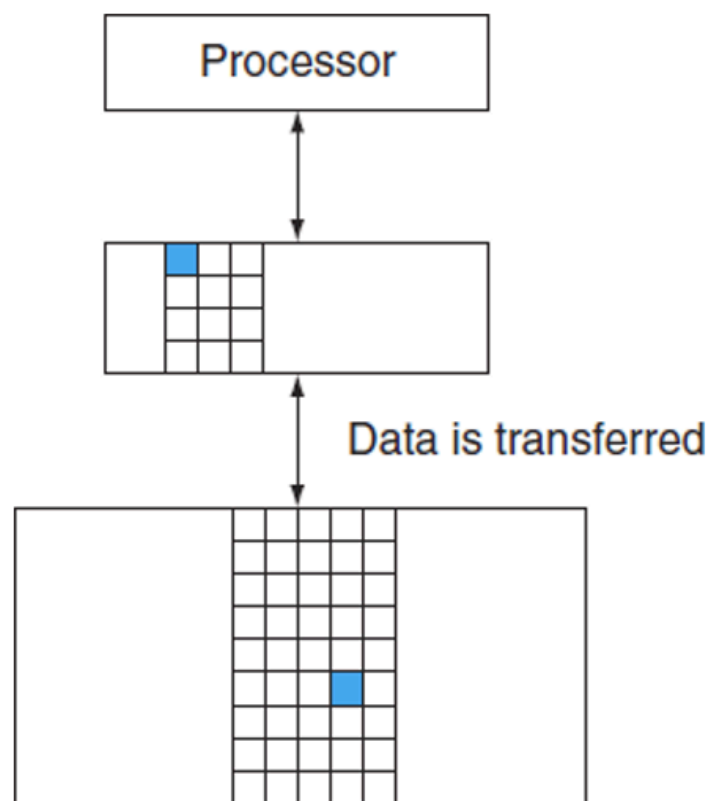
States that faster memories are more expensive per bit.

2. To take advantage of the "Principle of locality" by implementing the memory of a computer as a hierarchy.

3. Performance

Data hierarchy

- ↳ A level closer to the processor is a subset of any level further away & all the data is stored at the lowest level.
- ↳ Data is copied only between two adjacent levels of memory at a time.



- ↳ Minimum unit of information present in a cache is called Block.
- ↳ If data requested for the processor appears in some block in the upper level, this is called hit. If not, then the request is a miss.

Hit time:

The time to access the upper level of the memory hierarchy, which includes the time needed to determine whether the access is hit or miss.

Miss penalty:

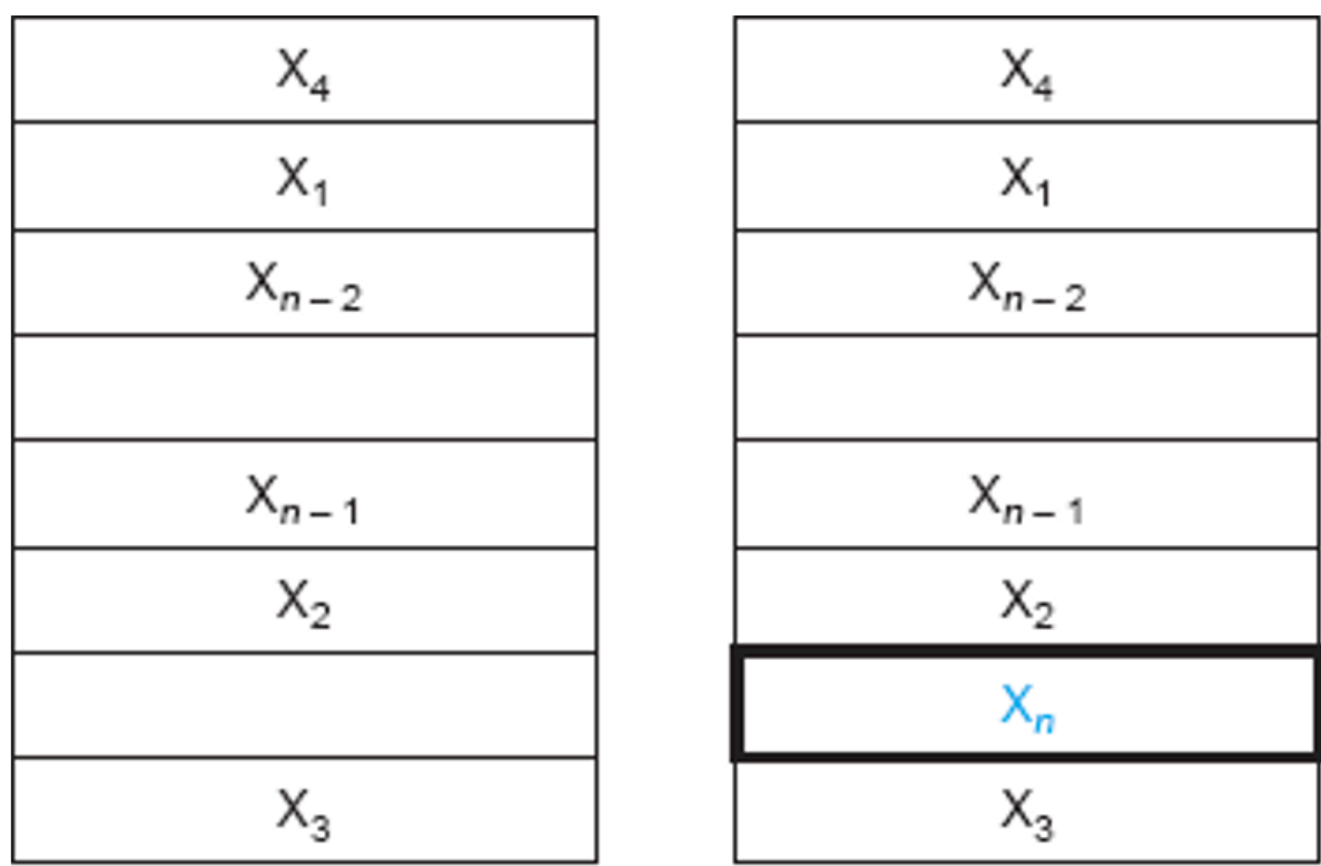
Time to replace a block in the upper level with corresponding block from the lower level plus the time to deliver this block to the processor.

Cache Basics

↳ Store main memory address + data at that address

Simple cache

Consider a cache in which the processor requests are each 1 one word & block is also one word.



a. Before the reference to X_n b. After the reference to X_n

NOTE:
why store block of data & not byte - in which processor requests for data?
This way we can take advantage of spatial locality

Direct mapped cache

↳ A cache structure in which each memory location is mapped to exactly one location in the cache, based on the address of the word in the main memory.

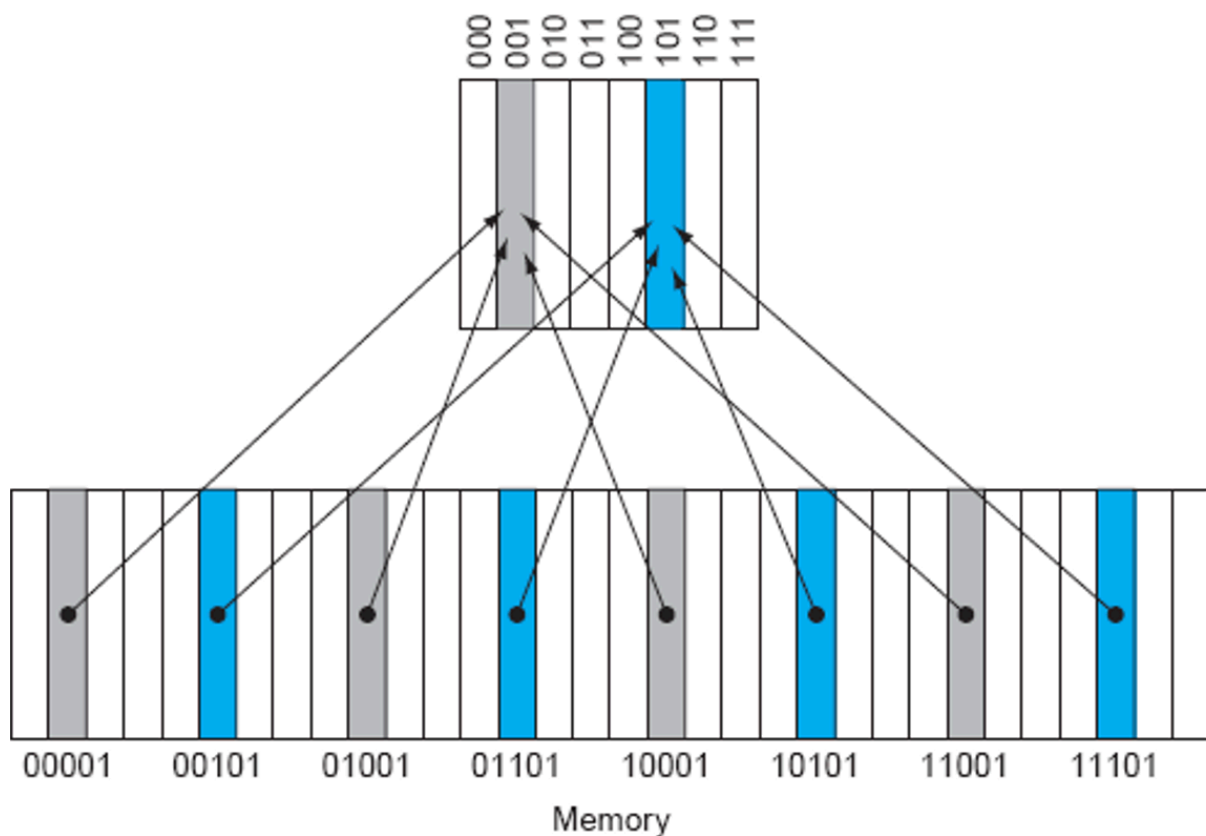
CACHE		
Tag	Index	Data

memory address is divided into two parts: Tag & index
Why?

When processor accesses data from cache, we match index first & if index is a match we check tag or else we move on.

Lower bits of address : $\log_2(\text{size of cache}) = \text{index}$

Remaining upper bits = Tag



Why do we need tag?

Assume cache size = 8
 so, index bits = $\log_3(8) = 3$

Now, 00001, 10001, 11001 all map to one index i.e 001 thus we need tag to be sure if data is actually in cache.

Example,

Sequence of memory references		
Address of reference	Assigned cache block	Hit or miss
10110	10 110 mod 8 = 110	miss
11010	11 010 mod 8 = 010	miss
10110	10 110 mod 8 = 110	hit
11010	11 010 mod 8 = 010	hit
10000	10 000 mod 8 = 000	miss
00011	00 011 mod 8 = 011	miss
10000	10 000 mod 8 = 000	hit
10010	10 010 mod 8 = 010	miss
10000	10 000 mod 8 = 000	hit

Cache		
Tag	Index	Data
10	000	M [10000]
	001	
10	010	M [11010]
00 10	011	M [00011] M [00011]
	100	
	101	
10	110	M [10110]
	111	

10011 10011 mod 8 = 011 miss

→ This is to illustrate

↳ In this case we replaced already existing data with this we got miss after matching tag.

$$\text{Miss rate} = \frac{5}{9}$$

= 55% Oops! So many!

Types of misses:

- Cold misses → No index matched
- Conflict misses: → index matched but not tag

Data Validity:

↳ When a processor starts up, caches does not have good data, and tag fields will be meaningless.

↳ Even after executing many instructions, some of the cache entries may still be empty.

↳ As such, we need to know that tag should be ignored.

Valid:

↳ A field to indicate whether the associated block in the hierarchy contains valid data.

↳ If the bit is not set, there cannot be a match for the required word.

Cache			
V	Tag	Index	Data
1			X4
1			X1
0			
0			Xn-2

Size of cache:

$$\text{Total \# bits} = \text{Total \# blocks} (\text{block size} + \text{tag size} + V\text{-bit})$$

Example,

How many total bits are required for a direct-mapped cache with **16 KB of data** and **4-word blocks**, assuming a 32-bit address?

$$1 \text{ word} = 4 \text{ bytes}$$

$$\begin{aligned} \text{block size} &= 4 \times 4 \times 8 \\ &= 128 \end{aligned}$$

$$\begin{aligned} \text{Num blocks} &= \frac{16 \times 1024 \times 8}{128} \\ &= 1024 \end{aligned}$$

$$\begin{aligned} \text{Index-size} &= \log_2(\text{num blocks}) \\ &= \log_2(1024) = 10 \end{aligned}$$

$$\begin{aligned} \text{word-offset} &= \log_2(\text{num-words-per-block}) \\ &= \log_2(4) = 2 \end{aligned}$$

$$\begin{aligned} \text{byte-offset} &= \log_2(\text{num-bytes-per-word}) \\ &= \log_2(4) = 2 \end{aligned}$$

$$\begin{aligned} \text{Tag size} &= \text{Total addr size} - \text{index size} - \text{word-offset} - \\ &\quad \text{byte-offset} \\ &= 32 - 10 - 2 - 2 \\ &= 18 \end{aligned}$$

$$\text{Total \# bits} = 1024 (128 + 18 + 1)$$

= 147kbits

Sources of cache miss

- Compulsory

- ↳ When process starts, cache is empty. So, every access to cache will be a miss

- Capacity:

- ↳ Cannot contain all blocks accessed by program

- ↳ Only a block of data is stored in cache locations, program may have more data/instruction than cache size. So, we can't save all due to less size of cache.

- Conflict,

- ↳ Multiple memory locations map to same location

- ↳ Index matches but tag doesn't

Fully Associative Cache

- ↳ Store anywhere

- ↳ Search each entry one by one when processor requests data.

Pro:

- flexible, always a spot will be found (no space is wasted)

- ↳ In Direct mapped, if all processor data access' address' index is same then all other locations will be un-used

Cons:

- Requires complex & expensive hardware

Set Associative Cache

- ↳ Cache is divided into sets
- ↳ Data's address determines which set to go into
 - ↳ Index is used as set & each set can have multiple data

Pros

- ↳ Best of fully associative & direct mapped caches
- ↳ Flexible
- ↳ Fast access time

Cons

- ↳ if all processor data access' address' index is same then all other sets will be un-used & one set will fill up (unlikely to happen)

Handling cache Misses

1. Send the original PC value to the memory.
2. Wait for the memory to complete its read access.
3. Write the cache entry, putting the data from memory in the data portion of the entry, writing the upper bits of the address into the tag field, and turning the valid bit on.
4. Re-fetch the instruction from the cache.

Handling writes to memory

↳ let your program

$X = X + 10$ { x is loaded from cache

When to write x to memory?

- **Write-through**

- A scheme in which writes always update both the cache and main memory, ensuring that data is always consistent between the two.

- **Write-back**

- A scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the main memory when the block is replaced.

Comparison

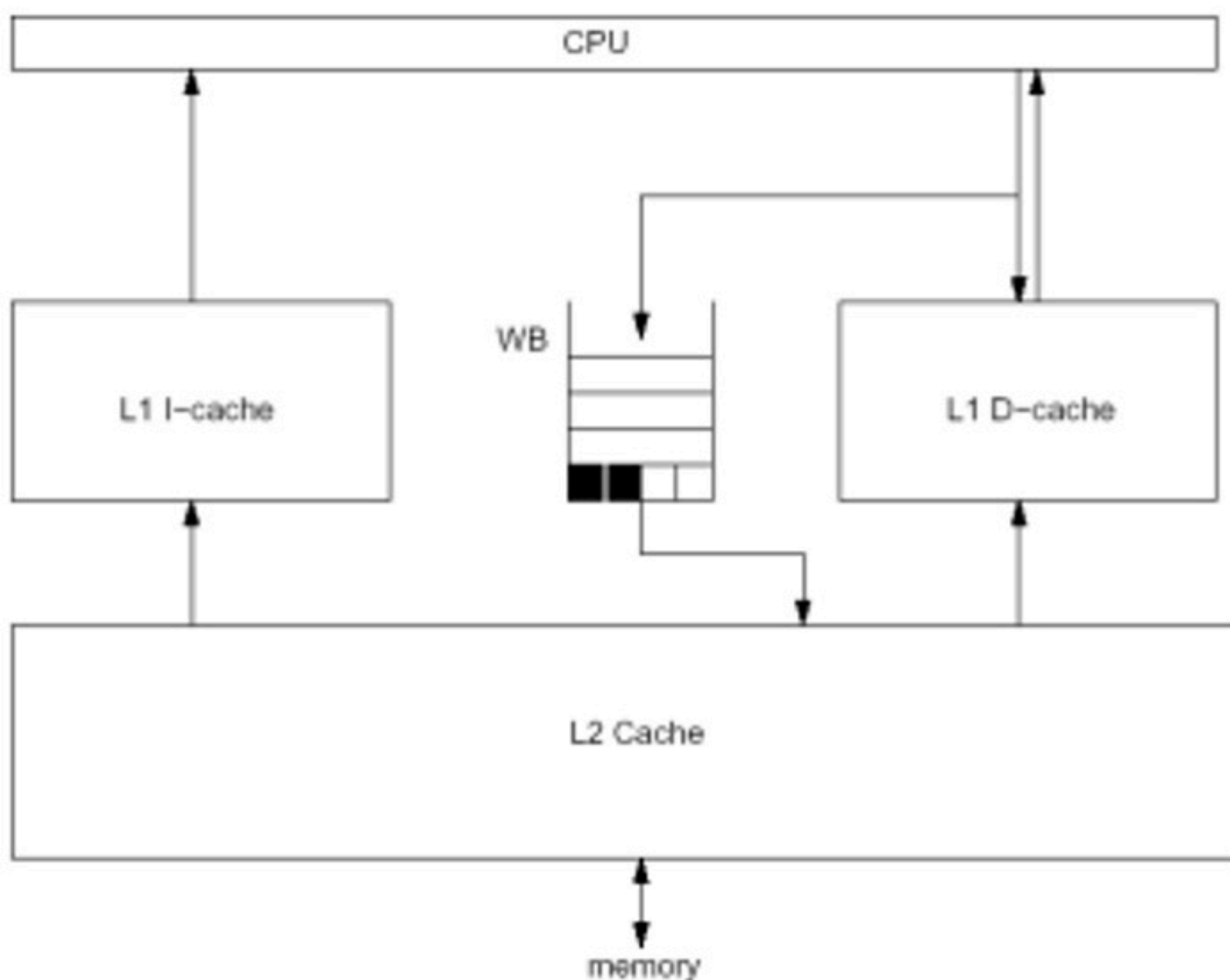
Write back

- Writes only go into top level of hierarchy
- Maintain a record of “dirty” lines
- Faster write speed (only has to go to top level to be considered complete)

Write through

- All writes go into L1 cache and then also write through into subsequent levels of hierarchy
- Better for “cache coherence” issues
- No dirty/clean bit records required
- Faster evictions

Instead of writing to memory direct, instead save to write buffer. Then when the buffer is full then write memory.



Cache coherence:

Cache coherence is the principle that ensures that multiple caches in a system, each storing a copy of the same memory location, are kept in sync.

It guarantees that all processors see a consistent view of memory despite the presence of local caches.

Maintaining cache coherence is essential for data consistency and program correctness.

MESI:

Every cache line (block) in a core's cache is tagged with one of four states:

M-Modified:

- ↳ Data in this is modified i.e it is different from main memory (dirty)

- ↳ If another core asks for this data, this cache must provide it & write it to memory

E-Exclusive:

- ↳ This cache has the only valid copy & it is clean.

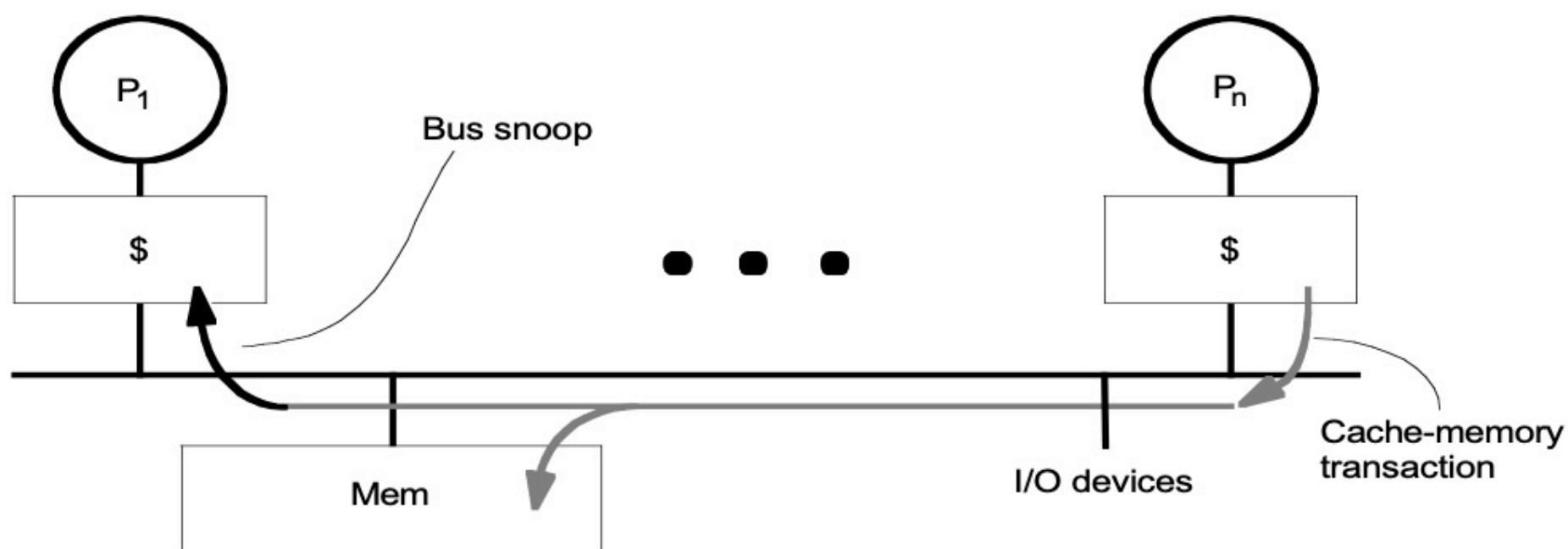
- ↳ If another core wants to change this, it must tell all other caches to invalidate their copies.

I-Invalid:

- ↳ This cache's line's data is not valid.

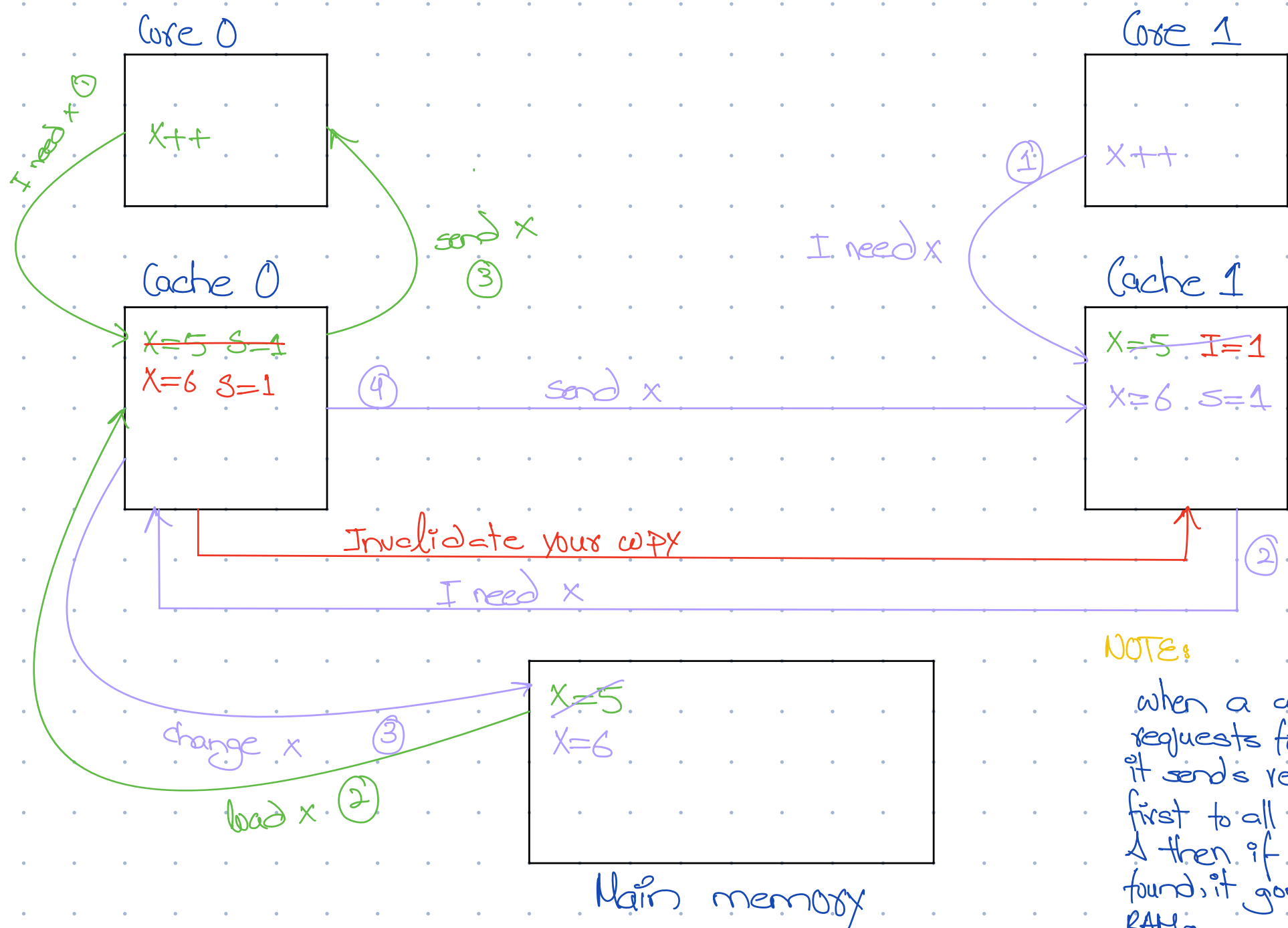
Cache coherence can be achieved via snoopy & directories

Snoopy:



When a core needs to access a memory location, it sends a search probe on the bus

All other cores listen to this message & perform necessary action i.e. send data to requesting core, invalidating its copy



NOTE:

when a core requests for data, it sends request first to all caches & then if not found, it goes to RAM.

Both cache's have x stored, where core 0 executes $x++$. x 's value is update in cache but not in memory, it also informs other caches

to invalidate their copy.

Now, when core 1 requests for x , it knows that its copy is invalid. So, it asks other caches for it.

At this, core 0's cache will provide x to it & write x to memory

↳ Contention on bus & thus more suitable for smaller-scale systems.

Initial State

Since both caches have a valid, identical copy of $x = 5$ that matches main memory, both cache lines start in the **Shared (S)** state.

Cache	data	M	E	S	I
Cache 0	5	0	0	1	0
Cache 1	5	0	0	1	0

 Export to Sheets 

Actually, there is a small misunderstanding in how the flags work. In the MESI protocol, these states are **mutually exclusive**. A cache line cannot be "Modified" and "Shared" at the same time.

When a bit is set to **1**, the others for that specific cache line must be **0**. Here is how the transition actually looks step-by-step:

1. After Invalidation (Before the Write)

When Core 0 signals its intent to write, Cache 1 must immediately mark its entry as **Invalid**. At this exact moment, Core 0 is technically the **Exclusive** owner because it's the only one with a valid copy, but it hasn't changed the data yet.

Cache	data	M	E	S	I	Notes
Cache 0	5	0	1	0	0	Now the sole owner; data matches memory.
Cache 1	5	0	0	0	1	Marked as "garbage."

 Export to Sheets 

2. After the Increment (The Final State)

Once Core 0 performs the addition ($5 + 1 = 6$), the state moves from **Exclusive** to **Modified**. This tells the system: "I have the only copy, and it is different from main memory."

Cache	data	M	E	S	I	Notes
Cache 0	6	1	0	0	0	Data is dirty; must be written back later.
Cache 1	5	0	0	0	1	Still invalid.

 Export to Sheets



Directory-based

There is a ^{centralized} directory associated with every memory block.

- ↳ Keeps track of copies of cached blocks & their states

- ↳ On miss, find directory entry, look it up, & communicate only to those caches

This approach is mainly used in larger multi-cores & multi-processor look-up

For instance

```
{
  "x": [stored by core 0's cache, exclusive/modified/shared, valid]
}
```

In your scenario where Core 0 increments x :

1. **Request:** Core 0 sends a private message to the Directory: "I want to write to address X."
2. **Filter:** The Directory looks at its ledger. It sees that **only Core 1** has a copy.
3. **Targeted Strike:** The Directory sends an invalidation message **only to Core 1**.
4. **Quiet Cores:** If there were a Core 2, Core 3, and Core 4, they would **never hear a single bit** of this conversation. Their caches remain undisturbed, and no bandwidth is wasted on them.

